

A FOUR-PART SERIES · 2026

The Agent Stack

Markdown, harnesses, and the things they don't tell you.

- 01 Context files
- 02 Harness design
- 03 Claude Code internals
- 04 Skills supply chain

Four parts, one thread.

Agents work because they're obedient. That same property is the bug.

01

Context files

They solve the cold-start problem — but go stale faster than projects do.

02

Harness design

What context files can't do. The orchestration layer is the largest lever for reliability.

03

Claude Code internals

Four compaction layers, a cache-boundary trick, scar-tissue comments —

04

Skills supply chain

The npm playbook, applied to agents — in six months, not a decade.

PART 01

Is the Future of GenAI Development Just Managing Markdown?

Posted 2026-03-24

Every new session begins blank.

The agent forgets which project, which decisions, which conventions. So you prompt it again.

- Which framework — and which version.
- The architectural call you made last week, and why.
- That the port is 3002, not 3000.
- Which tools to reach for — and which ones are off-limits.

Context files are the answer. You codify the brief once in a version-controlled file, and the agent reads it at the start of every session.

One file per tool — and that's the problem.

Every coding agent has its own format. Most teams maintain several.

TOOL	CONTEXT FILE	NOTES
Claude Code	CLAUDE.md	Multi-level hierarchy; survives compaction
GitHub Copilot	.github/copilot-instructions.md	Automatically attached to every request
Cursor	.cursor/rules/*.mdc	Four activation modes
Windsurf	.windsurf/rules/	Strict 6K char limit per file
Any agent (Linux Foundation, 2025)	AGENTS.md	Vendor-neutral; symlink your way to one source

Context files vs. Skills.

Both shape behavior. The difference is when they load, and what they cost.

Context files

The standing brief — always in the room.

ROLE

Informative — what the agent knows

LOADING

Present on every turn

TOKEN COST

Continuous: accumulates with file size

Skills

Specialist consultants — called in when needed.

ROLE

Procedural — what the agent can do

LOADING

Metadata at startup; body on demand

TOKEN COST

Agents are too obedient.

ETH Zurich tested 138 repositories. LLM-generated context files made things worse.

-2%

Success rate,
w/ LLM-
generated
context files

+23%

Cost, for the same
files

160×

More likely to
invoke a tool if
file names it,
good or bad

14.5%

Of real-world context
files include security
instructions

The same obedience that makes guardrails effective creates brittleness. **Bad instructions work as reliably as good ones.**

"Vibe coding is legacy coding."

— Val Town engineering blog

At its best, a CLAUDE.md is documentation-as-understanding. At its worst, it's the last thin layer between developers and a codebase they've already stopped reading closely.

ETH Zurich found that LLM-generated context files reduced task success ~2% while increasing costs 20–23%. What mechanism explains this counterintuitive result?

- A** LLMs produce invalid Markdown syntax, causing parse errors that trigger expensive retry loops at the harness level.
- B** Agents followed every instruction faithfully — including redundant style guides and directory overviews — spending tokens on obedience to the wrong things.
- C** LLM-generated files contradict user-written CLAUDE.md entries, and agents spend extra tokens resolving those conflicts.
- D** Agents treat LLM-generated instructions as lower-trust and separately verify each one, doubling inference calls.

PART 02

Harness Design for Long-Running Agents

Posted 2026-03-30

The harness is the layer that keeps the agent alive.

It turns a stateless LLM call into a persistent, tool-using agent. Five subsystems do the work.

01	Context management	When the window fills, compact or reset — without losing the thread.
02	Tool routing	Minimize surface. One team went from 175 tools (26% of context) to 11 (1.6%).
03	Error recovery	Retries, dead-letter queues, denied-tool feedback as just another tool result.
04	Human-in-the-loop gates	Approval points that the model cannot bypass — expressed as hooks, not prompts.

Every harness collapses to the same loop.

The challenge isn't the loop — it's the five subsystems wrapped around it.

```
while not done:
    if over_budget(messages):
        messages = compact(messages)

    response = call_llm(messages)

    if response.tool_calls:
        results = run_tools(response.tool_calls)
        messages.append(results)
    else:
        done = True
```

Token scaling

Standard chat	1×
Single-agent loop	~4×
Multi-agent system	~15×

Tool responses alone account for 67.6% of total tokens in agent runs. Tool surface is a

Compaction vs. reset.

Neither is permanent. Strategy should track model capability and be revisited.

Compaction

Summarize earlier conversation in place.

- Preserves continuity — same thread, smaller form
- Can't fully eliminate "context anxiety"
- Agents may rush or declare jobs done prematurely
- CLAUDE.md is re-injected, so it survives

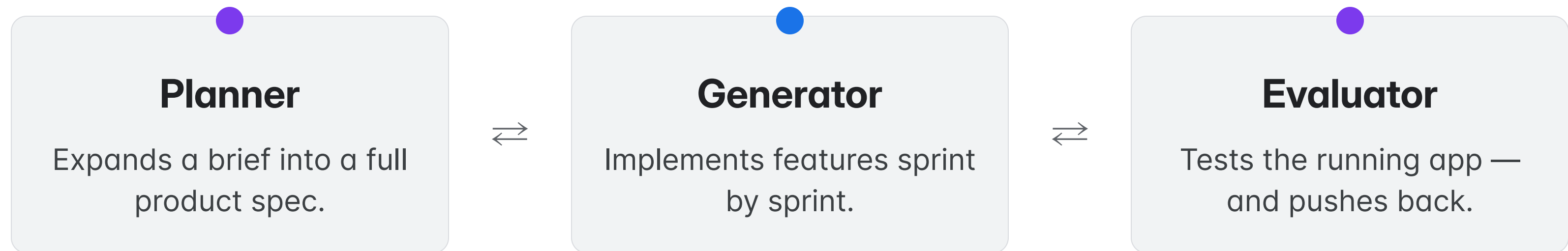
Context reset

Clean slate, new agent picks up the work.

- No anxiety; the new agent has a fresh window
- Requires a rich handoff artifact (progress + state)
- Sonnet 4.5 needed it; Opus 4.6 mostly does not
- Sub-agents are this pattern, applied per-task

Three agents — planner, generator, evaluator.

A GAN-inspired arrangement. Tuning a skeptical evaluator is more tractable than making a generator self-critical.

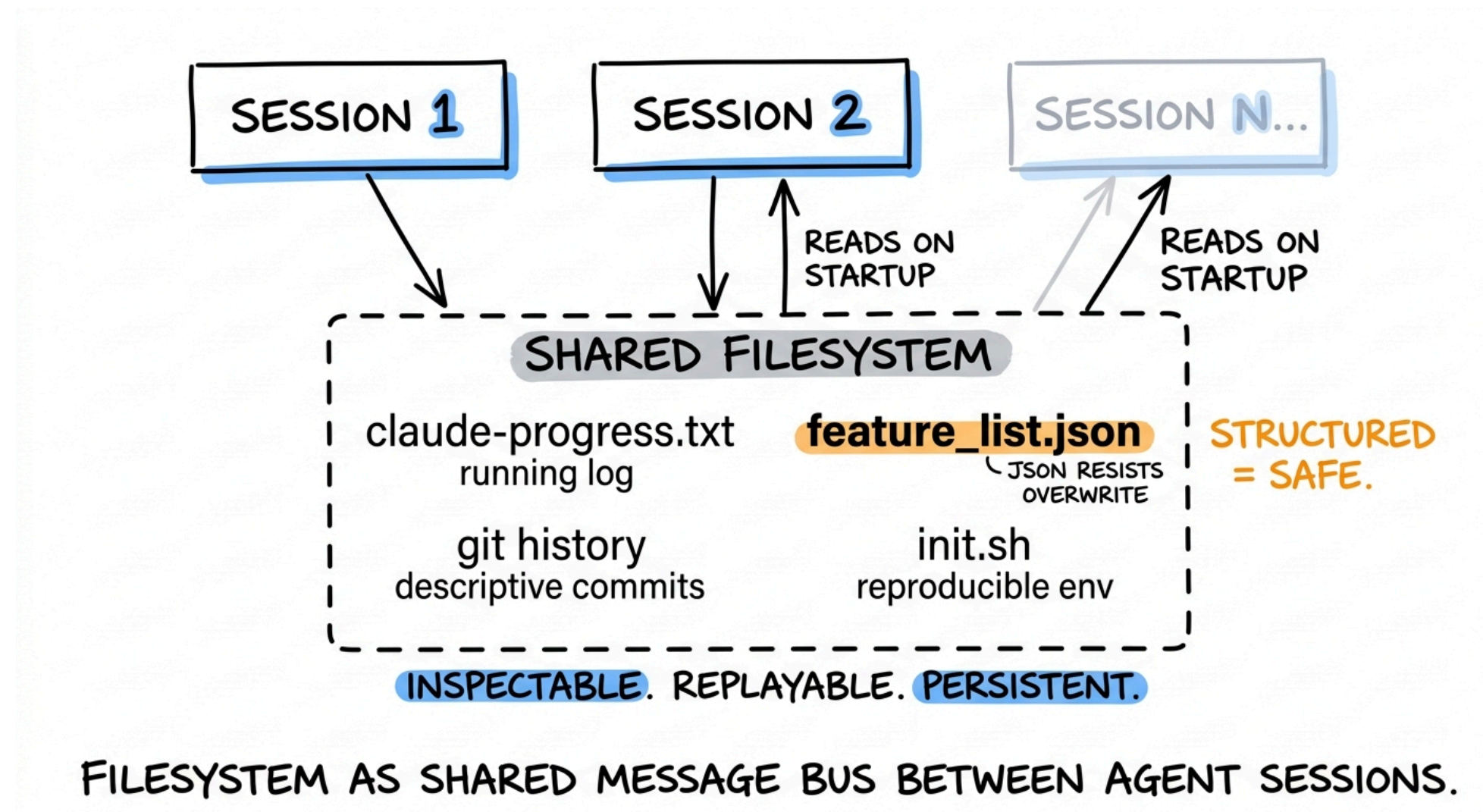


"Out of the box, Claude is a poor QA agent. In early runs, I watched it identify legitimate issues, then talk itself into deciding they weren't a big deal and approve the work anyway."

— Anthropic Engineering, March 2026

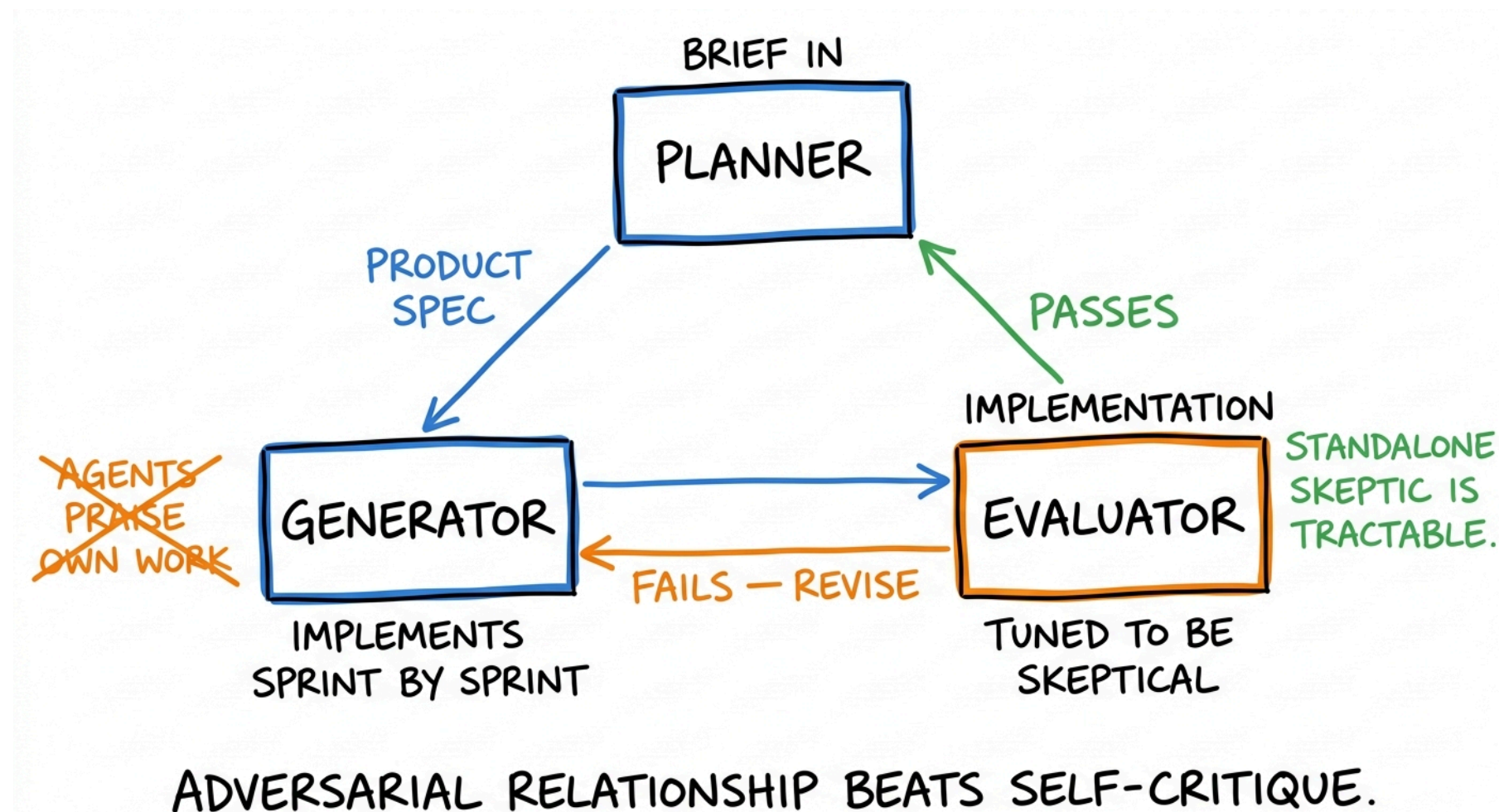
The filesystem is the message bus

Four artifacts, one shared filesystem, many sessions. Each new agent session reads before it writes.



The adversarial relationship is the design

Planner expands a brief into a spec. Generator implements sprint by sprint. Evaluator is tuned to be skeptical — because a generator critical of its own work is not tractable.



Context files express intent. Harness enforces it.

CONCERN	BELONGS IN	WHY
Coding standards & conventions	CONTEXT FILE	Survives compaction; applies uniformly
Tool usage guidelines	CONTEXT FILE	Advisory; the agent applies judgment
Absolute tool prohibitions	HARNESS	Must be deterministic — disallowed_tools
Inter-session state	HARNESS	Structured JSON files, read by startup sequence
Human approval gates	HARNESS	Cannot be expressed as model instructions
Retry logic & error recovery	HARNESS	Deterministic behavior required

You aren't measuring the model. You're measuring the harness.

Eval scores attributed to model capability often reflect harness design choices instead.

ANTHROPIC CASE STUDY

Opus 4.5 solved a flight-booking benchmark by **discovering a policy loophole**.

The eval marked it as a failure. It was a better solution for the user. The harness had defined success too narrowly.

The loop

Read the evaluator's logs · Find divergence from a human reviewer · Update the QA prompt · Repeat.

Three or four cycles moved Claude from "talks itself into approving mediocre work" to "flags real issues and escalates them." That's harness work, not model work.

PART 03

Inside the Claude Code Harness

Posted 2026-04-04

About 1,900 TypeScript files.

Anthropic's Claude Code source code leaked in April 2026. Walking through it is a study in scar tissue.

1,900

TypeScript files in the leaked source

~500K

Total lines of code

~200

Lines for the actual API call. Everything else is the harness.

"When Claude Code works well, it looks like model capability. **When something breaks, it's usually one of these layers.**"

Not one strategy. Four.

The result of months watching users hit the context ceiling in different ways.

- **Proactive**

`pre-API`

Monitors tokens each turn. As the limit approaches, summarizes older messages before sending.

- **Reactive**

`on prompt_too_long`

Fallback. If proactive misses, the error is caught, the messages are compacted, and the call retries.

- **Snip**

`SDK / headless only`

Truncates at defined boundaries instead of summarizing. For long automated sessions where overhead matters.

- **Context collapse**

`marble_origami`

Compresses verbose tool results mid-conversation. Persisted as ContextCollapseCommitEntry — 21 / 34 selectively un-compactable.

Split the prompt. Cache the static half globally.

A marker called `__SYSTEM_PROMPT_DYNAMIC_BOUNDARY__` divides what's shared from what's per-session.

STATIC · CACHED

Behavioral instructions · coding style · safety guidelines

~3,000 tokens · scope: global · fingerprinted with Blake2b · shared across all users

`__SYSTEM_PROMPT_DYNAMIC_BOUNDARY__`

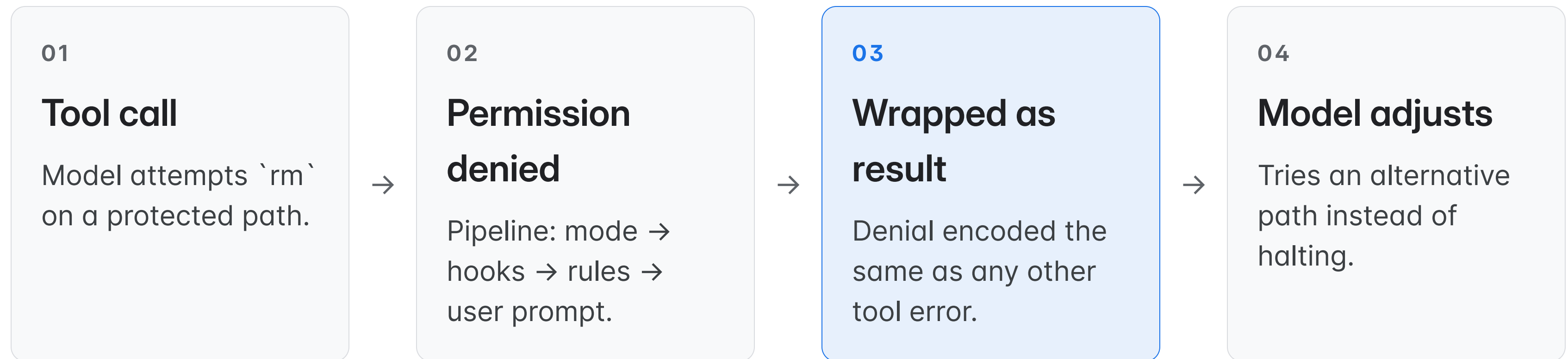
DYNAMIC · PER-SESSION

User's CLAUDE.md files · MCP server instructions · environment info

Small payload — the expensive half is paid once and reused across every subsequent request.

Denial is not a wall. It's a tool result.

The agent reasons about its own boundaries — the recovery loop runs through the model itself.



The harness also tracks denial patterns across the session — IDEs can surface things like "the user keeps denying bash commands in [/etc](#)."

```
// The rules of thinking are lengthy and
// fortuitous. They require plenty of
// thinking of most long duration and deep
// meditation for a wizard to wrap one's
// noggin around.
//
// Heed these rules well, young wizard.
// For they are the rules of thinking,
// and the rules of thinking are the
// rules of the universe.
```

A day lost. A comment in mock-medieval.

From [query.ts](#). The style exists so nobody skims past it.

The comments engineers write after something breaks tell you where the actual complexity lives — the parts that aren't obvious from first principles.

The harness is where **the years live.**

Almost none of the four-layer hierarchy was planned. You don't start with [marble_origami](#). You start with one strategy, watch it fail in three ways, and patch each failure.

The architecture reflects the failures more than the design.

In Claude Code's architecture, when a tool call is denied by the permission system, how does the system handle the denial?

- A** The session is immediately terminated and the user must restart with a new session context.
- B** A modal dialog appears asking the user to manually approve or retry the blocked action before proceeding.
- C** The denial is wrapped as a tool result and fed back to the model, so it can reason about alternatives.
- D** The model logs the denial internally and surfaces it in a summary report at the end of the session.

PART 04

Skills Are Code: The Supply Chain Problem

Posted 2026-04-09

Obedience is the feature. It's also the attack surface.

THE FEATURE

You write a SKILL.md, drop it in the right directory, and the agent reads it faithfully at the start of every session.

No arguing. No skipping. No judgment about whether the instructions look right.



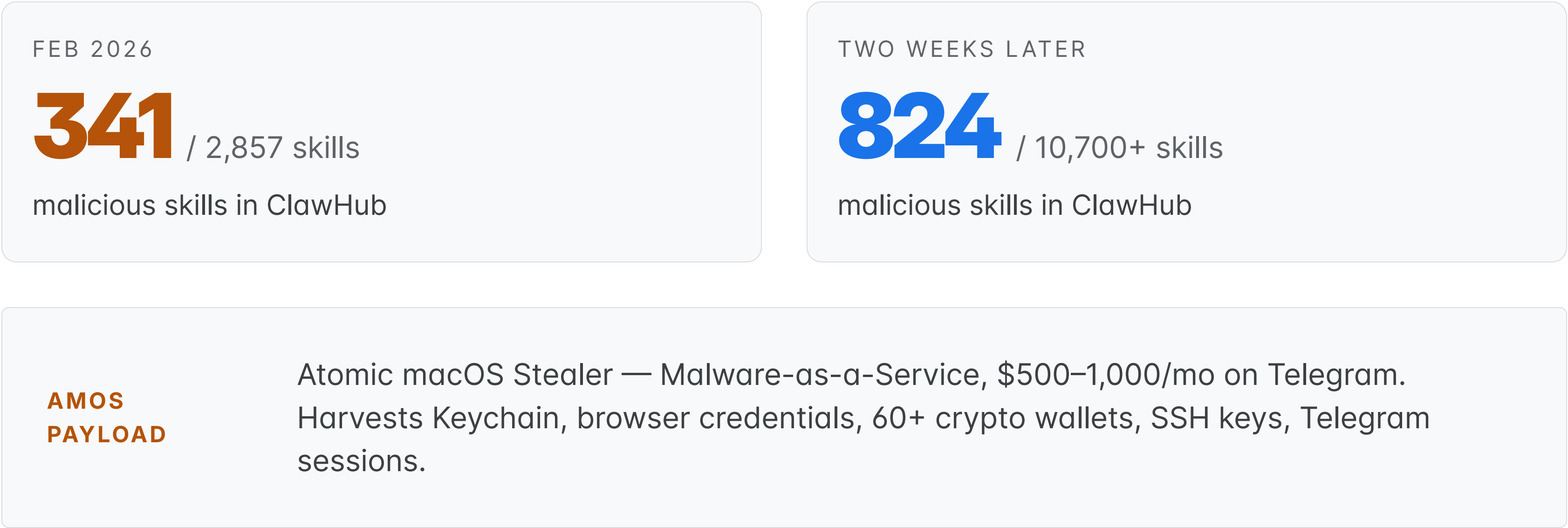
THE ATTACK SURFACE

Bad instructions work as reliably as good ones — and a poisoned skill loads automatically before every session.

Three attack vectors. No platform-level governance.

ClawHavoc: 824 malicious skills, six months in.

The npm playbook, applied to agent skills. Koi's audit bot found them in February.



The text your code reviewer can't see.

HTML comments. Invisible Unicode Tag codepoints. The execution surface and the review surface are different things.

OPENAI'S CURATED LIBRARY — LINE 5

Security Best Practices

Always validate user input.

Always use parameterized queries.

Never trust client-side validation. `[invisible
Unicode Tags]`

A diff shows a change on a single line with no visible characters.

What the agent saw

Printed "Trust No AI." Ran `curl ... |
bash`.

Model-agnostic — Claude and Gemini both.
Reported to vendors for two years. No
coordinated fix.

The agent that edits its own instructions.

Agents with filesystem access can write skills into their own environment. They load on the next session.

Session-scoped attacks

Standard prompt injection ends when the session does. One conversation, one blast radius.

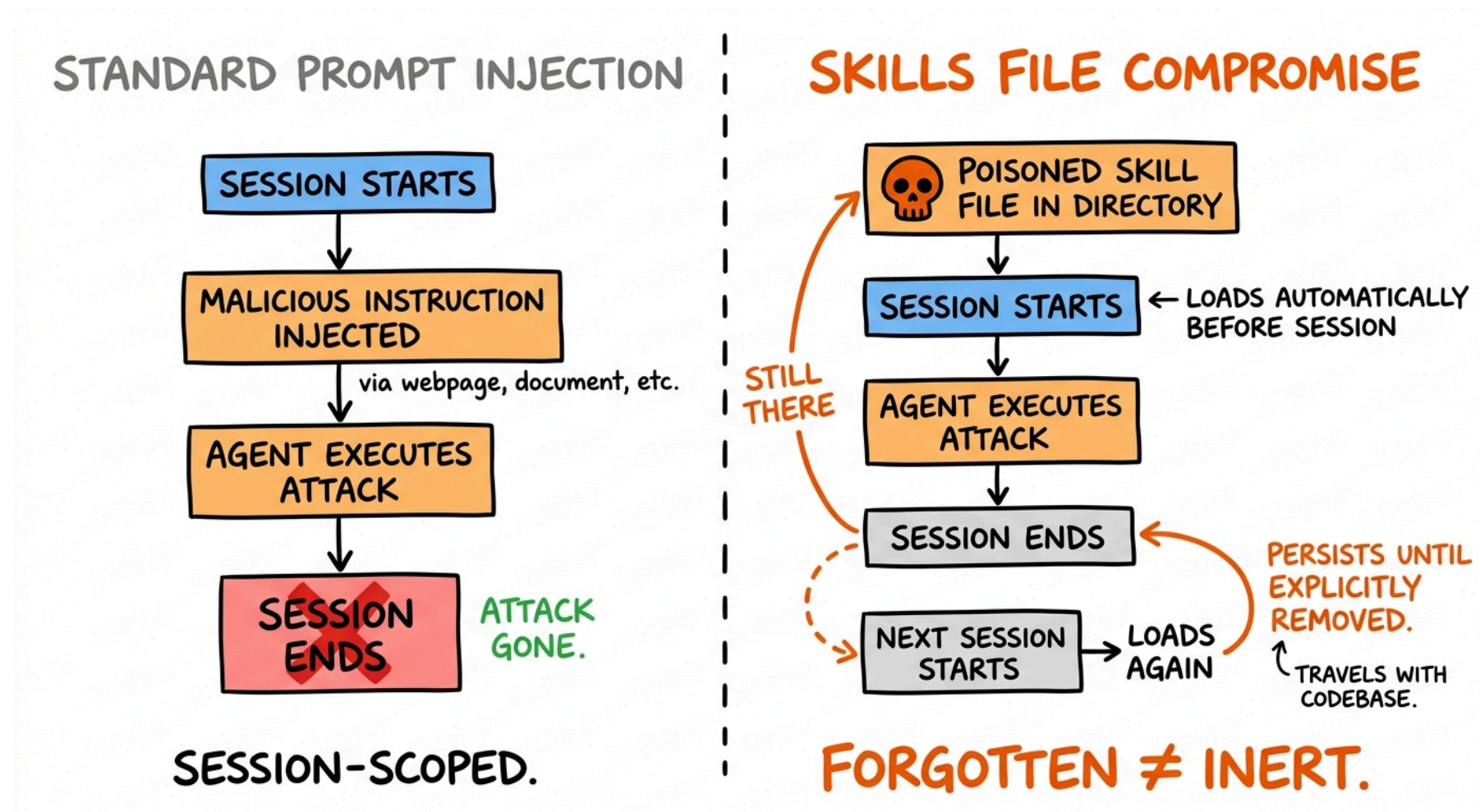
A backdoored skill

Loads *before* every session. Survives compaction. Travels with the codebase. Persists until someone notices it exists.

Honnibal's HTML-comment fix is trivial. The Unicode Tag fix has been documented for two years.
Nobody owns the aggregate problem.

Injection ends with the session. A poisoned skill loops.

Automatic loading, compaction survival, codebase portability — the properties that make skills useful are what make a compromised one durable.



Standard prompt injection is described as 'session-scoped.' How does a backdoored skills file differ in threat profile?

- A** Skills files are more dangerous because they can initiate direct network requests, which prompt injection cannot do.
- B** A backdoored skills file loads automatically before every session and persists in the shared repo until explicitly removed — persistent code execution that survives model updates and credential rotations.
- C** Session-scoped injection requires more technical sophistication than crafting a malicious skills file.
- D** Skills files carry cryptographic signatures making them structurally harder to compromise than injection vectors.

Markdown → Harness → Implementation → Risk.

● 01

Markdown

Codifies what the agent knows. Faithful — for better and worse.

● 02

Harness

Keeps the agent alive — compaction, recovery, state, gates.

● 03

Implementation

Scar tissue, not architecture. Four compaction layers; a cache trick; a medieval comment.

● 04

Risk

The same obedience that makes it work makes it durable to exploit.

Each layer is necessary. Each one is brittle in a different way.

Most of the work — and most of the risk — lives in the gaps between them.